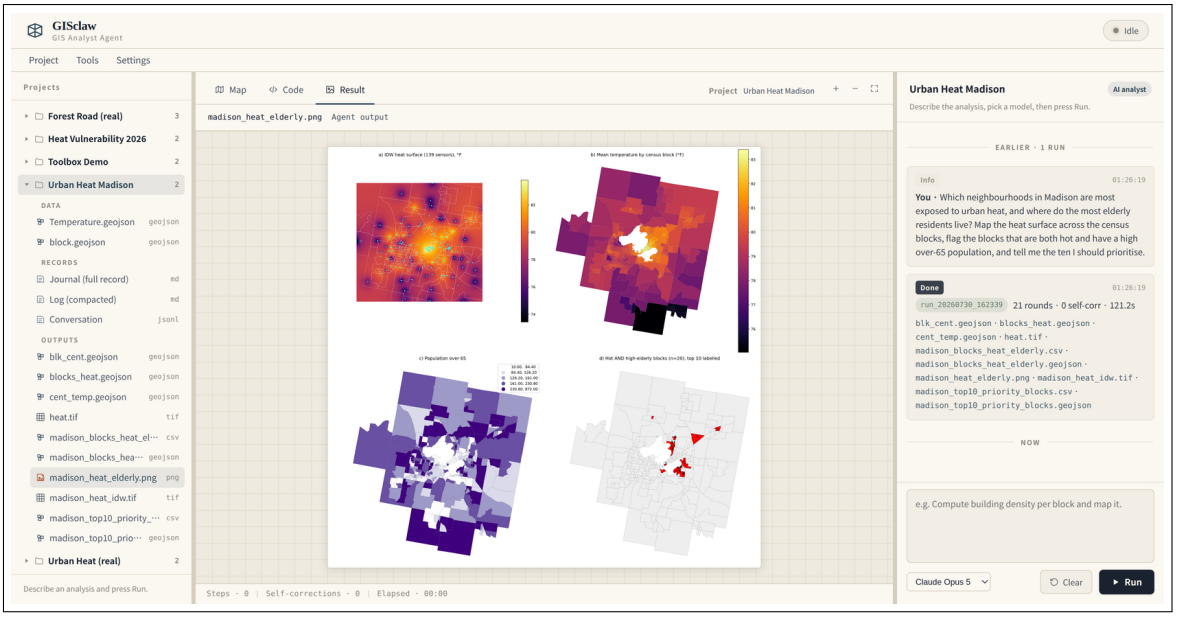
GISclaw

User Manual

Describe the spatial analysis you need. Get the finished work.



GISclaw is an automation tool for geospatial analysis. You state the question in ordinary language and it plans the workflow, runs it against your own data, and returns the layers, maps, tables, and a written account of how it got there. This manual covers installation, day-to-day use, the record it keeps, and the mechanisms that make a months-long project stay legible.

AGPL-3.0-or-later · Companion paper: arXiv:2603.26845 · v1.0

Contents

1 What GISclaw does **3**
1.1 Typical requests . . . 3

2 Installation **3**
2.1 Requirements . . . 3
2.2 Steps . . . 4

3 Daily use **4**
3.1 Creating a project and attaching data . . . 4
3.2 Writing a good request . . . 5
3.3 Watching a run . . . 5

4 How a run is governed **6**

5 What every project keeps **7**
5.1 The compacted log . . . 7

6 Different requests, different responses **8**

7 Standing preferences **8**

8 Skills **8**

9 The operations toolbox **9**

10 Where your data goes **9**

11 Troubleshooting **10**

12 Relationship to the research paper **10**

13 Licence and attribution **11**

A Setting up from scratch **12**
A.1 The three pieces, in plain terms . . . 12
A.2 Step 1 — Install Docker . . . 12
A.2.1 Windows 10 or 11 . . . 12
A.2.2 macOS . . . 13
A.2.3 Linux (Ubuntu, Debian and relatives) . . . 13
A.2.4 Confirm it works . . . 13
A.3 Step 2 — Get the GISclaw files . . . 13
A.3.1 Without git, the simplest route . . . 13
A.3.2 With git . . . 14
A.3.3 Opening a terminal inside that folder . . . 14
A.4 Step 3 — Obtain an API key . . . 14
A.5 Step 4 — Start GISclaw . . . 15

A.6 Step 5 — Your first analysis 15
A.7 Commands you will use regularly 15
A.8 If the setup does not work 16
A.9 Keeping the cost down 17

1 What GISclaw does

Spatial analysis is largely a sequence of well-understood operations assembled in the right order: reproject, interpolate, join, summarise, classify, map. The assembly consumes the hours, and it is where mistakes hide.

GISclaw accepts the request in the form a domain scientist would actually say it. The four-panel figure on the cover was produced from this single sentence:

> Which neighbourhoods in Madison are most exposed to urban heat, and where do the most elderly residents live? Map the heat surface across the census blocks, flag the blocks that are both hot and have a high over-65 population, and tell me the ten I should prioritise.

From that, GISclaw inspected both layers, found they used different coordinate reference systems, reprojected, built an IDW heat surface from 139 sensor readings, computed zonal means for 269 census blocks, normalised and combined heat with over-65 density, ranked the result, drew the figure, and wrote ten files. Twenty-one steps, 121 seconds.

1.1 Typical requests

You say	It produces
Where would flooding above 2 m cut off access to the hospital?	inundation extent, affected road segments, isolated catchments
How much protected forest falls within 500 m of the planned road?	buffers, intersections, affected area by protection class, an impact map
Which districts are underserved by fire stations?	service-area buffers, coverage gaps, population inside each gap
Compare forest cover between these two dates and quantify the loss.	change raster, gain/loss/net figures, a diverging-scheme map

2 Installation

2.1 Requirements

Docker and Docker Compose, roughly 3 GB of disk for the image, and an API key for one model provider. No GIS software needs to be installed: the container carries the full open-source geospatial stack.

> **New to Docker, GitHub, or model APIs?** The four steps below are a summary for readers who already have them. Appendix A walks through the same ground from nothing — installing Docker on each operating system, downloading the files without git, obtaining a key from each provider, and what to do when a step fails.

2.2 Steps

1. `git clone https://github.com/geumjin99/GISclaw.git` and `cd GISclaw`
2. `docker compose up -d`
3. Open `http://localhost:8765`
4. **Settings** → **API keys…**, paste a key, press **Test**

Where keys live. Keys are stored server-side under `./projects/.gisclaw/settings.json` with file mode `600`. They never enter the image, never reach the browser, and are returned to the interface only as a mask such as `sk-abc…wxyz`. Environment variables in a `.env` file work as a fallback for CI or shared hosts.

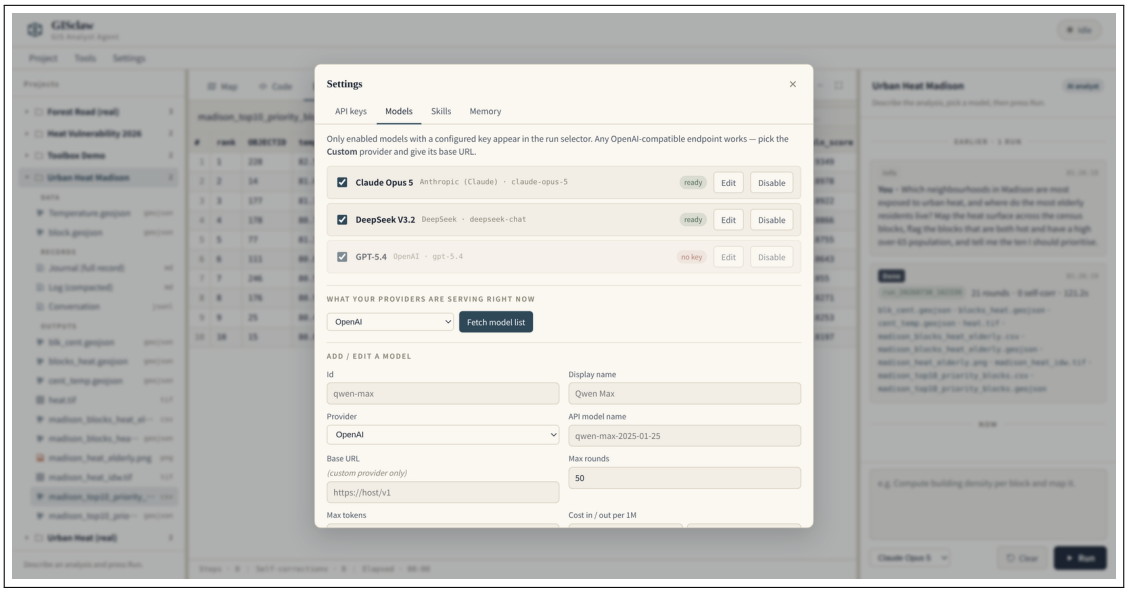


Figure 1: Model registry. The list is fetched live from each provider, so newly released models appear without an update to this software; only models with a working key are offered in the run selector.

3 Daily use

3.1 Creating a project and attaching data

A project is a working folder. Create one with **Project** → **New project…**, then attach data with **Project** → **Add data…**, which opens a server-side file browser rooted at the mounted workspace. You can also right-click a project folder in the tree.

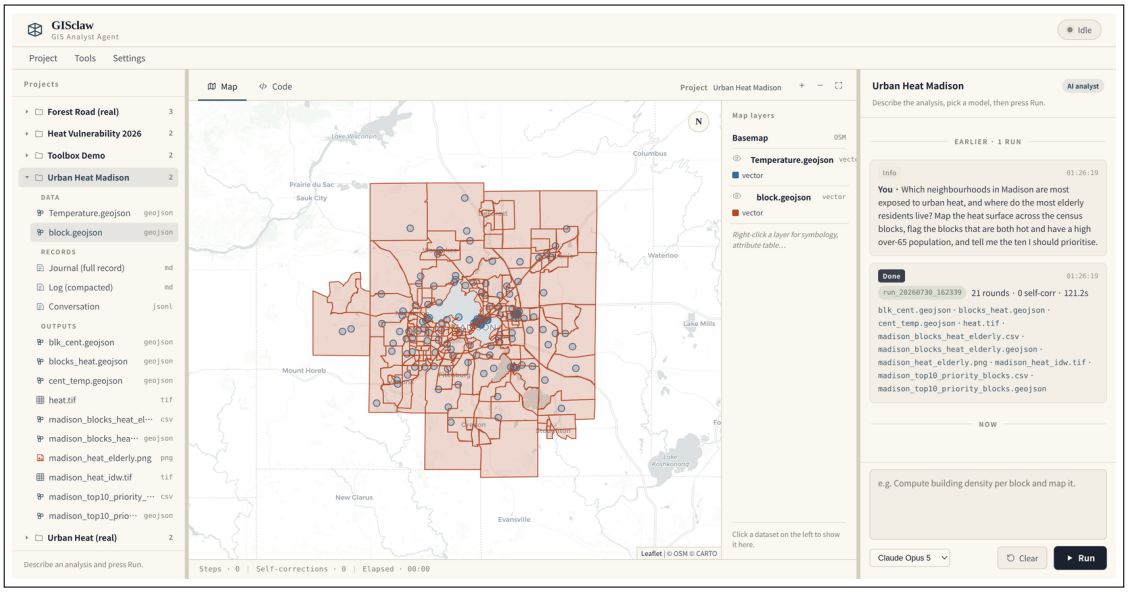


Figure 2: Layers rendered on the map, project tree on the left, conversation on the right. Right-click any layer for symbology, an attribute table, or zoom.

3.2 Writing a good request

State the goal, name any constraint that matters, and say what you want back. The agent reads the schema itself, so column names rarely need to be supplied.

Weak	Analyse the heat.
Workable	Interpolate the temperature points and summarise by block.
Strong	Interpolate the TemperatureF points into a continuous surface, compute the zonal mean for each census block, save blocks_meantemp.geojson and a choropleth PNG to pred_results/, and report the five hottest blocks and how many blocks have no value.

Asking for a number you can check — the five hottest blocks, the count with no data — is the cheapest way to catch a run that finished without being correct.

3.3 Watching a run

Thought, Action, and Observation stream into the conversation as they happen. Generated code appears in the **Code** tab, figures in **Result**, tables and text in **Table**, and vector or raster outputs render on the map automatically.

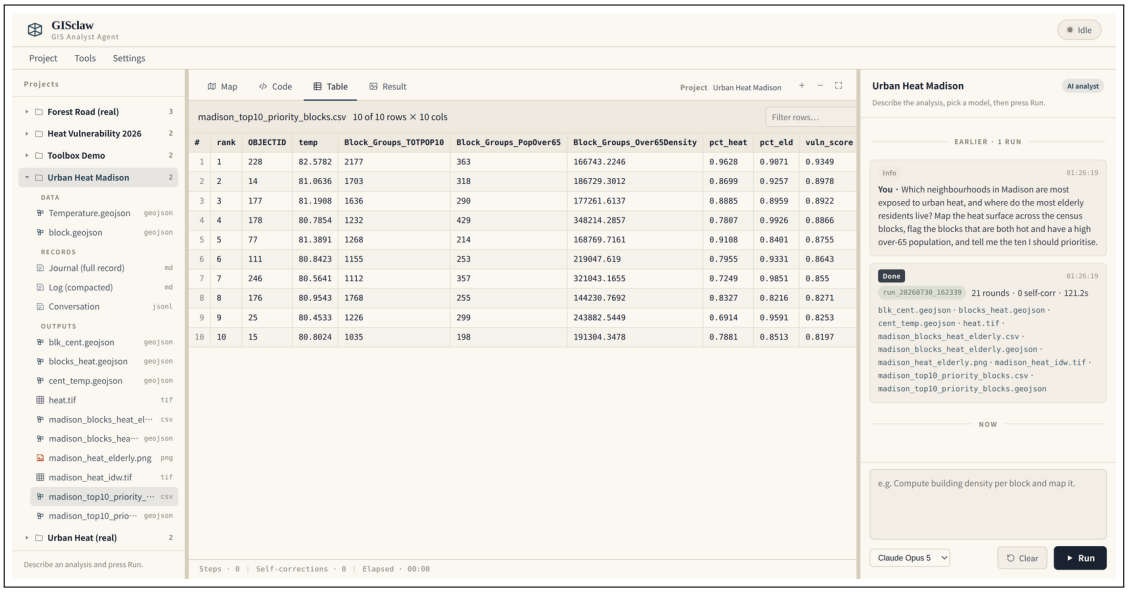


Figure 3: The request, the run summary with its outputs, and a produced table opened in the viewer.

4 How a run is governed

GISclaw drives a ReAct loop over a persistent Python sandbox: look at the data, reason about the next step, write a few lines, read the result, continue.

The operating discipline comes from roughly 1,800 controlled runs of this agent on real multi-step GIS tasks, and is applied to every analysis.

1. **Plan the whole path first.** The dominant failure is correct code for the wrong plan.
2. **Read the schema from the data.** Column names are never assumed.
3. **Treat CRS as a first-class concern.** Mixed projections are the norm; distance and area work happens in a projected system, and the null rate is printed after every join.
4. **Prefer the deterministic operations** over hand-written equivalents.
5. **Never fill in missing values quietly.** Units with no measurement stay null. Where filling is unavoidable, the reason, the count, and a flag column are required, and it must appear in the summary.
6. **Diagnose rather than retry.** The real error is at the end of a traceback.
7. **Verify before finishing.** Ranges, counts, and whether the mapped field varies at all.

Why rule 5 exists. A run that fills 170 of 269 units with the mean of the other 99 produces a complete-looking map in which two thirds of the pattern is manufactured. It looks like success in every automated check. Coverage of 99 units, stated plainly, is worth more.

5 What every project keeps

```
projects/<your project>/
├── data/                       what you attached
├── outputs/                    what came out
├── JOURNAL.md                  full record: every run, every round
├── LOG.md                      compacted digest, one entry per run
├── chat.jsonl                  the conversation, rebuilt in the UI
└── runs/run_<timestamp>/
    ├── code.py                 exactly what was executed
    ├── trace.jsonl             every Thought / Action / Observation
    └── pred_results/           that run's outputs, never overwritten
```

Refresh the browser, restart the container, or return in three months: the conversation is restored from `chat.jsonl`. Click any past run to replay its reasoning and load the code it executed.

> **Which copy to trust.** `outputs/` is a convenience copy that a later run may overwrite with the same filename. `runs/…/pred_results/` is never modified, so a specific result is always reproducible from there.

5.1 The compacted log

Long projects accumulate transcripts nobody rereads. After each run GISclaw spends one model call compressing the trace into five fields — Result, Method, Numbers, Caveats, Carries forward — and appends it to `LOG.md`. That digest is what feeds the next run's context.

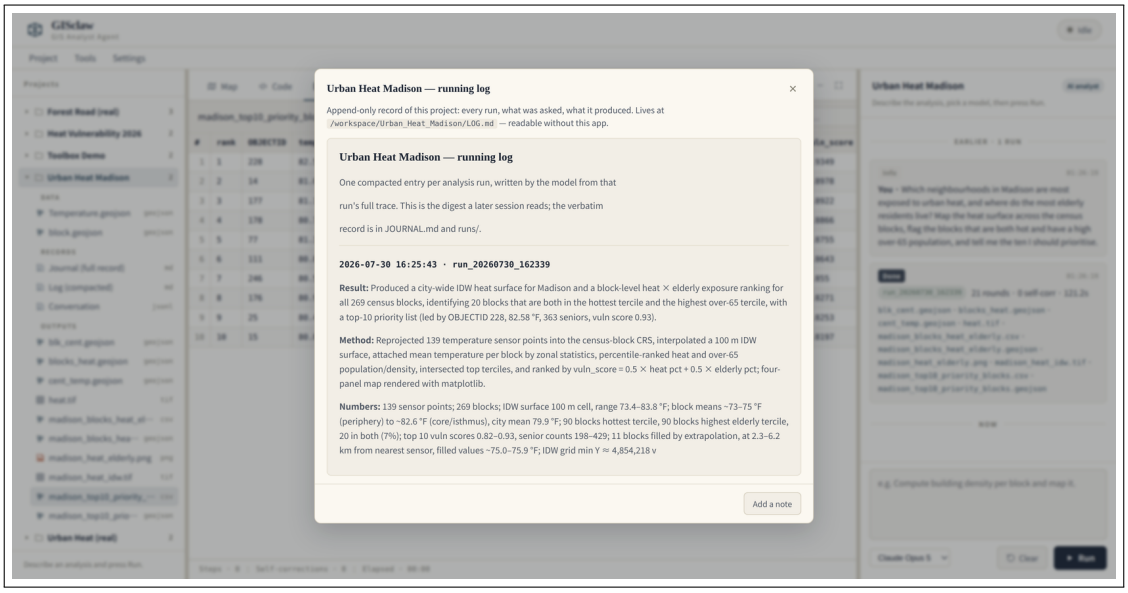


Figure 4: The compacted log. Each entry is written by the model from that run's full trace.

6 Different requests, different responses

GISclaw reads what kind of message it received before acting on it, which makes it usable through the whole arc of a piece of work rather than only at the moment you already know what to run.

- **A request for analysis** starts a run.
- **A question about the project** — what was done, what a layer holds, why a value looks odd — is answered from the record in a single call.
- **A proposal you want to think through** gets discussed. Sketch an approach, ask what it would do with the data you have, weigh two methods against each other, and settle the plan before any computation happens.
- **Ordinary conversation** stays ordinary.
- **Anything outside its scope** is declined with a note about what it does cover.

Discussing a method costs one short call; running it can cost several minutes and a dollar. Being able to agree on the approach first is often the cheaper path to the right answer.

7 Standing preferences

A global `MEMORY.md` holds conventions that apply to everything: colour ramps, the classification scheme you standardise on, the projected CRS for your region, what every deliverable map must carry. Write it once under **Settings** → **Memory…**, or hover any message and press **Remember**. It is injected into every run.

Keep it to rules. Text inside HTML comments is stripped before injection, so notes to yourself cost nothing.

8 Skills

A skill is a directory bundle that encodes how a class of analysis should be done:

```
<skill>/
├── SKILL.md          YAML frontmatter and a short router
├── manifest.yaml     optional declarative loading plan
└── references/*.md   the depth, opened only when a step calls for it
```

Loading is progressive, which keeps a large library affordable: a one-line description stays in context permanently (around 80 tokens), the router loads when the skill becomes relevant, and reference files open one at a time.

Level	Content	When it loads
Catalogue	name and one-line description	always
Router	`SKILL.md` body and file listing	when the skill is opened or matched
Depth	one reference file	when a step calls for it

Two skills ship with the application. `gis-analysis-discipline` carries the rules in section 4 and is always on. `gis-workflow-recipes` holds step-by-step templates for interpolation with zonal summary, multi-criteria suitability overlay, proximity and impact assessment, and two-date change detection.

Bundles import as a `.zip` or a folder, edit in the application, and export for sharing. Authors should declare `keywords:` in the frontmatter, which is what server-side matching uses to pre-load the right skill.

Figure 5: The skills panel, showing source, bundled file count, and the token cost each enabled skill adds.

9 The operations toolbox

Twenty-eight deterministic geoprocessing operations ship with the application: reproject, buffer, clip, intersection, difference, union, dissolve, spatial and attribute joins, zonal statistics, slope, aspect, hillshade, rasterize, IDW, and more.

They serve two practical purposes. They are tested and CRS-aware, so the agent reaches for them ahead of writing equivalent code by hand. And when you already know the exact operation you want, running it directly from the panel costs no API call at all, with the result rendered on the map immediately.

10 Where your data goes

Analysis runs inside the container against your native formats — Shapefile, GeoTIFF, GeoPackage, CSV, NetCDF. Conversion to GeoJSON or PNG happens only in the browser display layer, since that is what a browser can draw. The files stay on your machine. What reaches the model provider is your prompt and the agent's own observations of the data.

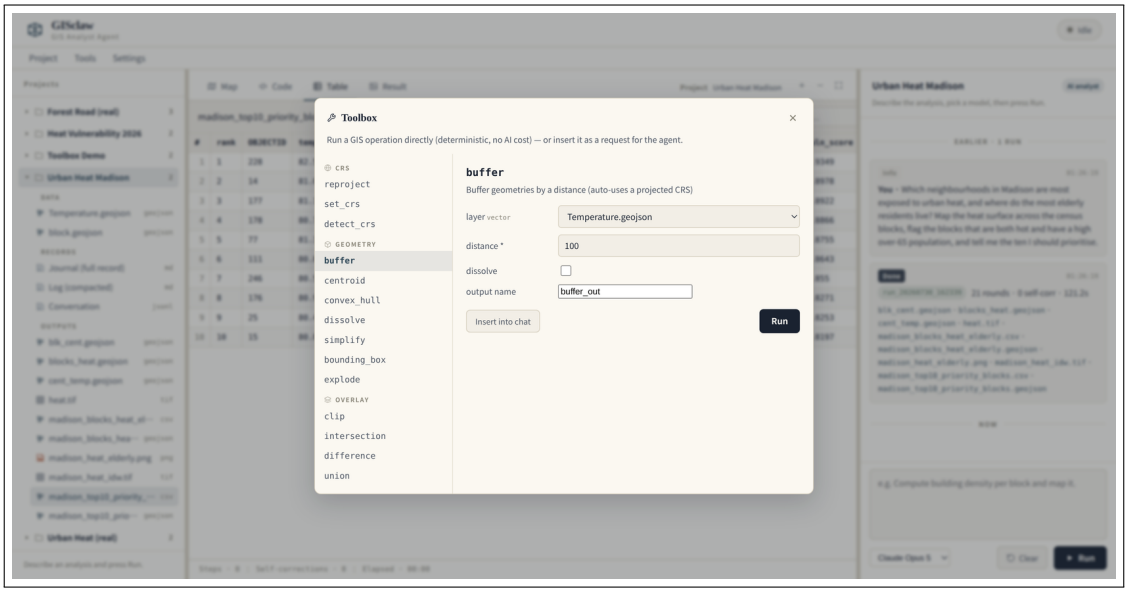


Figure 6: Each operation exposes a real parameter form. **Run** executes it deterministically; **Insert into chat** hands it to the agent as one step of a larger workflow.

11 Troubleshooting

Symptom	Check
Model selector reads No model configured	No provider has a key. Settings → API keys, then **Test**.
A run fails immediately with a key error	The selected model's provider has no key; choose a model marked ready.
Raster operations fail on the host	Run rasters inside the container; a broken host PROJ database affects `rasterio.warp` but not vector work.
The interface looks stale after an update	Hard-refresh the browser and confirm the console reports the expected `app.js` version.
Intermediate files appear in `outputs/`	Each geoprocess step writes to `pred_results/`; intermediates are copied along with final products.

Server logs: `docker logs -f gisclaw_app`. Per-run logs: `projects/<project>/runs/<run_id>/run.log`.

12 Relationship to the research paper

The system described in arXiv:2603.26845 is a benchmark harness built for controlled experiments. This desktop application was rebuilt on that agent core and has since diverged substantially: an operations toolbox usable from the interface, project persistence with journal and compacted log, skill bundles with progressive disclosure, global preference memory, and live model discovery.

The experimental results reported in the paper came from the research harness at its frozen commit. Reproduction work should use that artifact.

13 Licence and attribution

A Setting up from scratch

This appendix assumes you have never used GitHub, never used Docker, and have never obtained an API key. Worked through in order, it takes you from a plain computer to a running copy of GISclaw. Set aside about an hour for the first attempt; most of that is waiting for downloads, not typing.

A.1 The three pieces, in plain terms

Docker
The geospatial libraries GISclaw needs — GDAL, GEOS, PROJ, GeoPandas, rasterio — are famously painful to install correctly, and a wrong version quietly breaks coordinate handling. Docker sidesteps the problem: you download one prepared box with everything already inside and working, and it stays isolated from the rest of your computer.

The GISclaw files
A folder of code and configuration, published on GitHub. GitHub is simply where the folder is hosted; you do not need an account and you do not need to understand version control to download it.

An API key
GISclaw performs the geospatial work itself, but the reasoning comes from a large language model running on a provider's servers. The key is a long secret string that identifies your account with that provider and bills usage to you. There is no GISclaw account, subscription, or licence fee.

Nothing else is required. You do not need QGIS, ArcGIS, or a Python installation.

A.2 Step 1 --- Install Docker

Windows 10 or 11

1. Download **Docker Desktop** from `https://www.docker.com/products/docker-desktop/` and choose the build matching your processor (almost always **AMD64**; pick ARM64 only for a Snapdragon-based machine).
2. Run the installer and leave **Use WSL 2 instead of Hyper-V** ticked.
3. Restart the computer if you are asked to.
4. Start Docker Desktop and wait until the whale icon stops animating and the status reads Engine running.

The usual blocker is hardware virtualisation being switched off in the BIOS or UEFI. Docker Desktop will say so plainly; the setting is called Intel VT-x, AMD-V, or SVM Mode depending on the motherboard.

macOS

Download Docker Desktop from the same address and take care to choose the build for your chip: **Apple Silicon** for M-series machines, **Intel** for older ones. The wrong build will not start. If you are unsure, check **Apple menu** → **About This Mac**. Drag the application to Applications, launch it, and approve the privileged helper when macOS asks.

Linux (Ubuntu, Debian and relatives)

The `docker.io` package shipped by most distributions is too old for the `docker compose` syntax used here. Install from Docker's own repository by following `https://docs.docker.com/engine/install/`, then allow your user to run Docker without `sudo`:

```
sudo usermod -aG docker $USER
```

Log out and back in for that to take effect.

Confirm it works

Open a terminal and run these three commands:

```
docker --version
docker compose version
docker run --rm hello-world
```

The third downloads a tiny test image and prints Hello from Docker!. If it does, your installation is sound and the rest of this appendix will work.

> **`docker-compose` with a hyphen is the obsolete version 1.** This manual uses `docker compose` with a space. If only the hyphenated form exists on your machine, upgrade before continuing — version 1 does not understand this project's configuration.

A.3 Step 2 --- Get the GISclaw files

Either method below works, and neither requires a GitHub account.

Without git, the simplest route

1. Visit `https://github.com/geumjin99/GISclaw`.
2. Click the green **Code** button, then **Download ZIP**.
3. Unpack the archive somewhere you can find again — your Documents folder is fine. You will get a folder named `GISclaw-main`.

Prefer a path without spaces or non-English characters. Both usually work, but they are the first thing to rule out if something later behaves oddly.

With git

```
git clone https://github.com/geumjin99/GISclaw.git
cd GISclaw
```

The advantage is that `git pull` later fetches updates without downloading everything again.

Opening a terminal inside that folder

Every command from here on must run inside the folder that contains `docker-compose.yml`.

- **Windows** — open the folder in File Explorer, click into the address bar, type `cmd` and press Enter. Or right-click empty space in the folder and choose **Open in Terminal**.
- **macOS** — right-click the folder and choose **Services** → **New Terminal at Folder**. If that entry is missing, enable it under **System Settings** → **Keyboard** → **Keyboard Shortcuts** → **Services**. Alternatively, open Terminal, type `cd ` and drag the folder onto the window.
- **Linux** — right-click inside the folder and choose **Open in Terminal**.

Confirm you are in the right place by listing the contents — `ls` on macOS and Linux, `dir` on Windows. You should see `docker-compose.yml` among the files.

A.4 Step 3 --- Obtain an API key

You pay the model provider directly for what you use. Costs are modest: a single analysis typically falls somewhere between a fraction of a cent and a few tens of cents, depending on the model and how long the task runs.

Any one of these is enough to start:

- **DeepSeek** — `https://platform.deepseek.com`
  By a wide margin the cheapest, and entirely adequate for learning the software. A good first choice.
- **Anthropic (Claude)** — `https://console.anthropic.com/settings/keys`
  Claude Opus 5 is GISclaw's default and the strongest option for long multi-step analyses.
- **OpenAI** — `https://platform.openai.com/api-keys`
  Widely used and well documented.
- **Google Gemini** — `https://aistudio.google.com/apikey`
  Offers a free tier, which is useful for a first run at no cost.

The procedure is the same everywhere:

1. Create an account on the provider's console at the address above.
2. Add a payment method. Most providers require some credit on the account before the first call will succeed; some grant a small trial balance.
3. Find the section named **API keys** or **Credentials**.
4. Create a new key and give it a recognisable name such as `GISclaw`.
5. **Copy it immediately.** Providers display the full key exactly once. If you lose it, delete that key and create another — there is no way to view it again.

Treat a key like a bank card. Anyone holding it can spend your balance. Never paste one into a chat message, a screenshot, a bug report, or a file you commit to a repository. If a key is ever exposed, revoke it in the provider's console straight away and issue a new one. GISclaw itself keeps keys server-side in `./projects/.gisclaw/settings.json` with file mode `600`, and only ever returns a mask such as `sk-abc…wxyz` to the browser.

Provider consoles are redesigned often. If the labels above have moved, look for wording along the lines of API keys, Credentials, or Billing.

A.5 Step 4 --- Start GISclaw

From the terminal you opened in Step 2:

```
docker compose up -d
```

The first run builds the image and is slow. Docker fetches the geospatial stack from conda-forge and installs the Python packages, finishing at roughly 1.8 GB. Expect ten to thirty minutes on an ordinary connection, with long stretches where nothing appears to happen — that is normal, and it is downloading. Every later start takes a few seconds, because the built image is reused.

When the command returns, open `http://localhost:8765` in a browser.

Go to **Settings** → **API keys…**, paste the key from Step 3 next to its provider, and press **Test**. A successful test makes that provider's models available in the run selector. This is also the explanation if the model list looks empty: only models backed by a working key are offered.

A.6 Step 5 --- Your first analysis

A worked example ships with the software. The application can only see files under the mounted workspace, which is the `projects` folder next to `docker-compose.yml`, so copy the example in first:

```
cp -r examples/urban-heat-madison projects/
```

On Windows, in the same folder:

```
xcopy /E /I examples\urban-heat-madison projects\urban-heat-madison
```

Then, in the browser: **Project** → **New project…**, name it, **Project** → **Add data…** and select the two GeoJSON files, and type a request such as map summer heat exposure by census block and show me which blocks are worst. Watch the trace on the right as it works.

A.7 Commands you will use regularly

All of them run in the folder holding `docker-compose.yml`.

Command	What it does
`docker compose up -d`	Start GISclaw in the background
`docker compose stop`	Stop it, keeping the container and its image
`docker compose down`	Stop and remove the container; your `projects/` folder is untouched
`docker compose logs -f`	Follow the server log; press Ctrl+C to stop watching
`docker compose restart`	Restart, which is what to do after changing settings files
`docker compose up -d --build`	Rebuild the image after updating the source

Your work never lives inside the container. Projects, data, outputs, logs and keys are all in the `projects` folder on your own disk, so removing and recreating the container loses nothing.

A.8 If the setup does not work

This table covers problems getting GISclaw to start. For difficulties during an analysis, see the Troubleshooting section earlier in this manual.

What you see	What to do
`docker: command not found`	Docker is not installed, or not on the PATH. Reinstall; on Windows and macOS confirm Docker Desktop is actually running.
`Cannot connect to the Docker daemon`	The engine is not running. Start Docker Desktop, or on Linux `sudo systemctl start docker`.
`permission denied` on `docker.sock` (Linux)	Your user is not in the `docker` group. Run `sudo usermod -aG docker $USER`, then log out and back in.
`env file .env not found`	Your Docker Compose predates version 2.24. Either upgrade it, or create an empty file named `.env` in the same folder.
`port is already allocated`	Something else holds port 8765. Stop that program, or edit `docker-compose.yml` to read `"8766:8765"` and use `http://localhost:8766` instead.
The build fails part-way while downloading	Almost always the network. Run `docker compose up -d` again — completed layers are cached, so it resumes rather than restarting.
The page will not load	The container may have exited. Run `docker compose logs` and read the last lines.
The model list is empty	No key has passed **Test** yet. Return to Step 3.
The interface looks out of date	Force a reload with Ctrl+Shift+R, or Cmd+Shift+R on macOS.

A.9 Keeping the cost down

- Begin with DeepSeek while you are learning the software; switch to a stronger model once the request matters.
- One well-specified request costs less than several vague ones, because the agent spends fewer rounds discovering what you meant.
- Reuse what exists. Asking to add a column to a layer produced earlier is far cheaper than recomputing it, and GISclaw is told about previous outputs so it can do exactly that.
- Set a monthly spending limit in the provider's console. Every provider offers one, and it is the only hard guarantee against a surprise.
- Each run records its own cost. **Project** → **Log (compacted)** and **Journal** both show what has been spent.